PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Muhammad Daffa Al Barru

NPM : 25782017

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

MENTRI PENDIDIKAN DAN KEBUDAYAAN PROVINSI LAMPUNG

POLITEKNIK NEGERI LAMPUNG

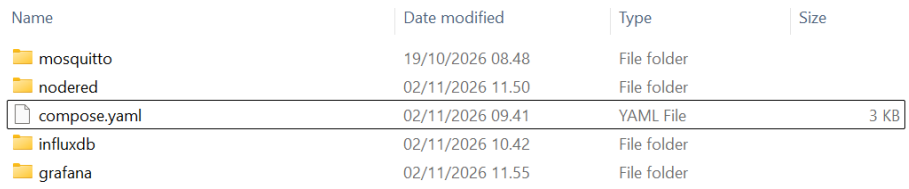
TAHUN AJARAN 2025 / 2026

8.5 Praktikum

8.5.1 Praktikum 1: Peningkatan Ekosistem IoT Server dengan InfluxDB v2 dan Grafana pada Docker Compose

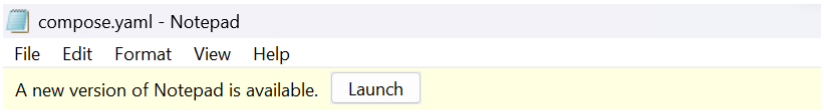
Pada praktikum ini, kita akan memperluas konfigurasi server IoT lokal yang telah dibuat pada Modul 7 dengan menambahkan kontainer **InfluxDB versi 2** (sebagai basis data deret waktu) dan kontainer **Grafana** (sebagai persiapan antarmuka visualisasi industri untuk Modul 9).

1. Buat subdirektori penyimpanan fisik persisten baru untuk InfluxDB dan Grafana:



Name	Date modified	Type	Size
mosquitto	19/10/2026 08.48	File folder	
nodered	02/11/2026 11.50	File folder	
compose.yaml	02/11/2026 09.41	YAML File	3 KB
influxdb	02/11/2026 10.42	File folder	
grafana	02/11/2026 11.55	File folder	

2. Buka berkas **docker-compose.yml** di dalam folder iot-server menggunakan editor teks (VS Code atau Notepad). Perbarui seluruh isinya dengan dokumen konfigurasi empat layanan terpadu berikut:

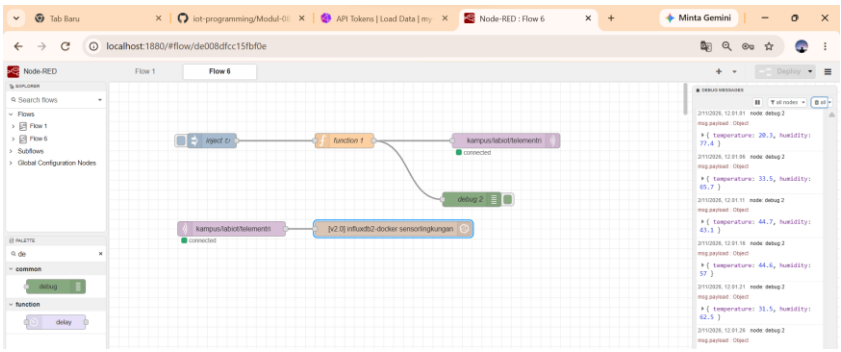



```
# ============================================================
# 3. Layanan Basis Data Deret Waktu: InfluxDB v2
# ============================================================
influxdb:
  image: influxdb:2.7
  container_name: influxdb2
  restart: unless-stopped
  ports:
    - "8086:8086"  # Port HTTP API & Web UI InfluxDB v2
  environment:
    # Inisialisasi otomatis akun administrator dan bucket awal
    DOCKER_INFLUXDB_INIT_MODE: setup
    DOCKER_INFLUXDB_INIT_USERNAME: admin
    DOCKER_INFLUXDB_INIT_PASSWORD: adminpassword
    DOCKER_INFLUXDB_INIT_ORG: my-org
    DOCKER_INFLUXDB_INIT_BUCKET: my-bucket
    DOCKER_INFLUXDB_INIT_RETENTION: 30d
  volumes:
    - ./influxdb/data:/var/lib/influxdb2
    - ./influxdb/config:/etc/influxdb2

# ============================================================
# 4. Layanan Dasbor Visualisasi Industri: Grafana
# ============================================================
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"  # Port Antarmuka Dasbor Web Grafana
  environment:
```

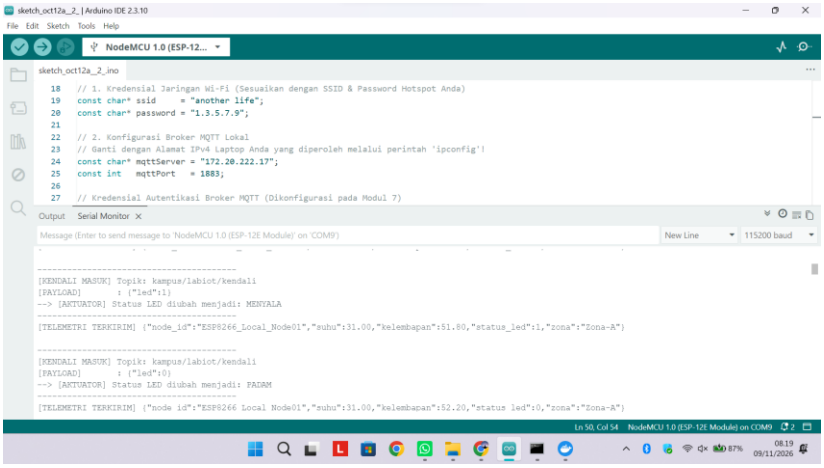
3. Simpan berkas docker-compose.yml. Jalankan perintah berikut untuk mengunduh *image* InfluxDB dan Grafana serta menghidupkan seluruh ekosistem kontainer di latar belakang (*detached mode*):
4. Verifikasi Status Kontainer:
   Jalankan perintah berikut untuk memastikan keempat kontainer telah berstatus Up:

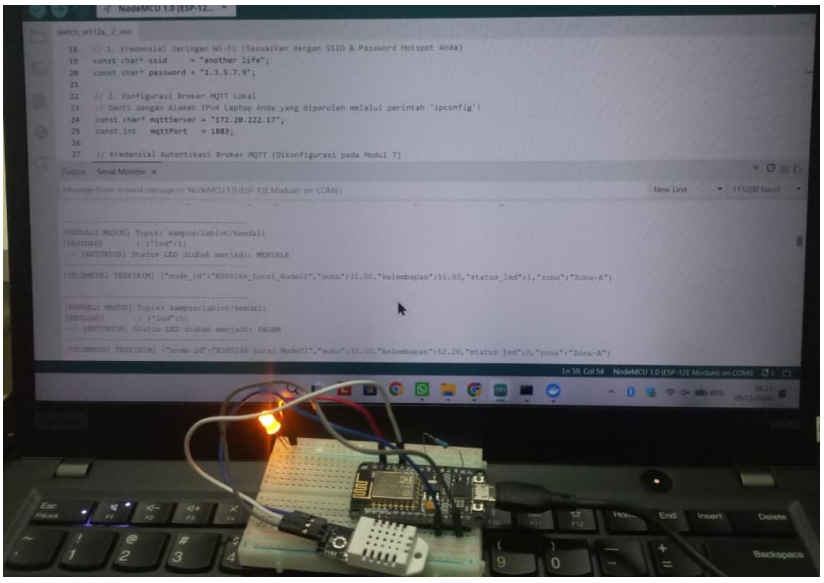
```
C:\DOCKER\iot-server>docker compose up -d
[+] up 4/4
 ✔Container influxdb2   Running                                                
```

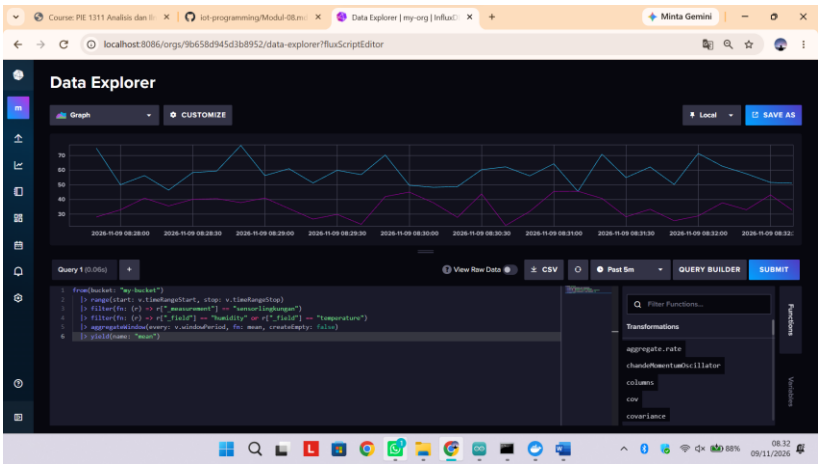


8.5.4 Praktikum 4: Integrasi End-to-End NodeMCU ESP8266 dan Verifikasi Telemetri Time-Series

kita akan memprogram NodeMCU ESP8266 agar membaca sensor fisik DHT11/22, menyematkan metadata "zona", mengirimkannya ke broker lokal, lalu memverifikasi data yang masuk secara visual melalui antarmuka **Data Explorer** di InfluxDB v2.







8.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Kinerja & Efisiensi Penyimpanan: TSDB vs RDBMS):** Sebuah pabrik cerdas memasang 100 simpul sensor NodeMCU ESP8266. Setiap simpul mengirimkan data telemetri yang terdiri dari 3 metrik *float* (suhu, kelembapan, getaran mesin) setiap 2 detik secara kontinu selama 30 hari penuh.

    i. Hitung jumlah total titik data (*data points*) yang harus disimpan oleh sistem basis data selama periode 30 hari tersebut! Jumlah total titik data yang harus disimpan adalah **388.800.000 data points**. Hasil ini diperoleh dari perhitungan: 100 simpul sensor × 1.296.000 kali pengiriman dalam 30 hari (30 hari × 24 jam × 60 menit × 60 detik / 2 detik) × 3 metrik per pengiriman

    ii. Jika menggunakan RDBMS konvensional dengan indeks *B-Tree*, rata-rata ukuran per baris transaksi adalah sekitar 150 byte. Sedangkan pada InfluxDB v2 dengan kompresi *Gorilla* dan *Delta*, ukuran per titik data terkompresi hanya sekitar 2.5 byte. Berapa rasio penghematan ruang penyimpanan (*storage savings*) yang dihasilkan oleh InfluxDB dibandingkan RDBMS? Rasio penghematan ruang penyimpanan yang dihasilkan oleh InfluxDB dibandingkan RDBMS adalah sebesar **98,33%** (menghemat penyimpanan lebih dari 60 kali lipat). Hal ini didasarkan pada total ukuran data RDBMS konvensional yang mencapai 58,32 GB (388.800.000 baris × 150 byte), sedangkan pada InfluxDB v2 terkompresi hanya membutuhkan sekitar 0,972 GB (388.800.000 titik data × 2,5 byte).

    iii. Jelaskan mengapa struktur indeks *B-Tree* pada RDBMS mengalami penurunan performa drastis (*write degradation*) saat menangani laju data

konstan tersebut! Struktur indeks *B-Tree* pada RDBMS dirancang untuk pencarian acak seimbang, sehingga saat menerima laju data konstan berkecepatan tinggi dari ratusan sensor, sistem dipaksa untuk terus-menerus melakukan penyisipan baris dan penyeimbangan ulang pohon (*tree rebalancing*). Proses ini memicu fragmentasi memori sekunder dan *disk I/O bottleneck*, yang semakin diperparah oleh *overhead* transaksi ACID serta penguncian baris (*row locking*).

- **Latihan 2 (Fenomena Cardinality Explosion pada InfluxDB Tags):**
  Seorang pengembang IoT pemula memasukkan nilai acak presisi tinggi seperti nomor transaksi UUID (uuid=a8f9-c412...) atau nilai pembacaan sensor pecahan (suhu_str=29.84512) ke dalam **Tag Set** InfluxDB, bukan ke dalam *Field Set*.

  i. Jelaskan bagaimana mekanisme *Inverted Index* pada InfluxDB bekerja terhadap *Tags* dan mengapa tindakan pengembang tersebut memicu fenomena **Cardinality Explosion**! Mekanisme *Inverted Index* pada InfluxDB memetakan setiap nilai *Tags* ke dalam indeks pencarian di memori utama, di mana setiap kombinasi unik dari *measurement* dan *tags* membentuk satu *series key*. Ketika pengembang memasukkan data acak presisi tinggi (seperti nomor UUID atau nilai desimal pecahan) ke dalam *Tag Set*, InfluxDB memperlakukan tiap nilai unik tersebut sebagai *series key* baru, yang memicu pembengkakan kombinasi deret waktu secara eksponensial (*Cardinality Explosion*).

  ii. Apa dampak fatal fenomena *Cardinality Explosion* terhadap konsumsi memori utama (RAM) peladen dan stabilitas kontainer Docker InfluxDB? Dampak utamanya adalah konsumsi memori utama (RAM) peladen yang melonjak drastis karena seluruh *Inverted Index* wajib disimpan di dalam RAM agar pencarian cepat. Ketika memori fisik habis terpaksa melampaui batas, sistem operasi atau Docker Engine akan menghentikan kontainer InfluxDB secara mendadak melalui mekanisme *Out of Memory (OOM) Killed*, sehingga basis data mengalami *crash* dan lumpuh total.

  iii. Rumuskan pedoman baku bagi seorang insinyur sistem untuk menentukan kapan sebuah variabel data wajib dijadikan *Tag* dan kapan wajib dijadikan *Field*! Sebuah variabel wajib dijadikan **Tag** jika nilainya berupa metadata kontekstual berjenis string diskrit dengan jumlah variasi terbatas (*low cardinality*) yang sering digunakan untuk penyaringan atau pengelompokan kueri (contoh: node_id, lokasi, atau zona). Sebaliknya, variabel wajib dijadikan **Field** jika nilainya berupa metrik numerik hasil pengukuran fisik,

data kontinu, atau string unik acak yang nilainya selalu berubah pada setiap sampel (contoh: suhu, kelembapan, getaran, atau UUID).

- **Latihan 3 (Arsitektur Jaringan Docker dan Resolusi Nama Layanan DNS):** Pada Praktikum 3, di dalam konfigurasi node influxdb out pada antarmuka Node-RED, kita mengisi alamat server dengan URL **http://influxdb:8086**, sedangkan pada peramban web laptop kita mengakses InfluxDB melalui **http://localhost:8086**.

    i. Mengapa penulisan URL http://localhost:8086 di dalam konfigurasi node influxdb out pada Node-RED akan menyebabkan kegagalan koneksi (*Connection Refused / ETIMEDOUT*)? Penulisan http://localhost:8086 di dalam kontainer Node-RED menyebabkan kegagalan koneksi (*Connection Refused* atau *ETIMEDOUT*) karena kata kunci localhost (127.0.0.1) merujuk pada ruang jaringan terisolasi (*network namespace*) milik kontainer Node-RED itu sendiri. Karena layanan basis data InfluxDB berjalan pada kontainer terpisah dan tidak mendengarkan port 8086 di dalam kontainer Node-RED, koneksi langsung ditolak.

    ii. Jelaskan peran *Docker Embedded DNS Resolver* (127.0.0.11) dan jaringan *bridge* (iot-net) dalam memfasilitasi komunikasi antar-kontainer tanpa harus mengekspos port ke jaringan publik! Jaringan *bridge* (iot-net) menyediakan jaringan virtual privat agar antar-kontainer dapat saling berkomunikasi secara aman tanpa membuka port ke publik. Docker Embedded DNS Resolver (127.0.0.11) bertindak sebagai pelayan DNS internal yang secara otomatis menerjemahkan nama servis deklaratif (seperti http://influxdb:8086) menjadi alamat IP internal kontainer InfluxDB yang dinamis di dalam jaringan iot-net

- **Latihan 4 (Strategi Retensi Data & Agregasi Berjenjang / Downsampling):** Dalam arsitektur IoT industri, data telemetri berfrekuensi tinggi (sampel per detik) sangat penting untuk investigasi cepat jika terjadi gangguan mesin dalam 7 hari terakhir. Namun, untuk pelaporan kinerja tahunan (*annual KPI*), manajemen hanya membutuhkan nilai rata-rata per jam.

    i. Mengapa menyimpan data berpresisi detik selama bertahun-tahun dianggap sebagai pemborosan sumber daya komputasi (*inefficient architecture*)? Menyimpan data presisi detik selama bertahun-tahun dianggap tidak efisien karena membebani media penyimpanan fisik (*disk*) dengan miliaran baris data mentah yang jarang diakses. Selain itu, kueri analitik jangka panjang (seperti laporan tahunan) menjadi sangat lambat karena basis data harus

memindai dan memproses komputasi data berukuran besar yang sebenarnya tidak relevan untuk analisis tren.

ii. Jelaskan konsep **Retention Policy** pada *Bucket* InfluxDB v2! *Retention Policy* pada *Bucket* InfluxDB v2 adalah aturan durasi penyimpanan data otomatis yang menentukan berapa lama data historis akan dipertahankan. Ketika usia data melampaui batas retensi yang disetel (misalnya 7 hari atau 30 hari), InfluxDB secara otomatis menghapus blok data usang tersebut dari sistem berkas tanpa memerlukan kueri DELETE manual yang berat.

iii. Rancang strategi arsitektur dua *bucket* (*Dual-Bucket Strategy*): jelaskan bagaimana bahasa skrip Flux dapat diprogram sebagai tugas berkala (*InfluxDB Task*) untuk melakukan *downsampling* otomatis dari bucket_raw_7d ke bucket_aggregated_1y! Strategi dua *bucket* membagi penyimpanan menjadi bucket_raw_7d untuk menyimpan data mentah presisi detik dengan retensi pendek (7 hari) dan bucket_aggregated_1y untuk menyimpan data ringkasan dengan retensi panjang (1 tahun). Skrip Flux diprogram sebagai tugas berkala (*InfluxDB Task*) menggunakan fungsi aggregateWindow (misalnya menghitung rata-rata per jam every: 1h, fn: mean) untuk membaca data dari bucket_raw_7d, meringkasnya, lalu menuliskan hasilnya secara otomatis ke bucket_aggregated_1y.

8.7 Tugas: Implementasi Metadata Tagging Multi-Zona dan Pemfilteran Kueri Telemetri

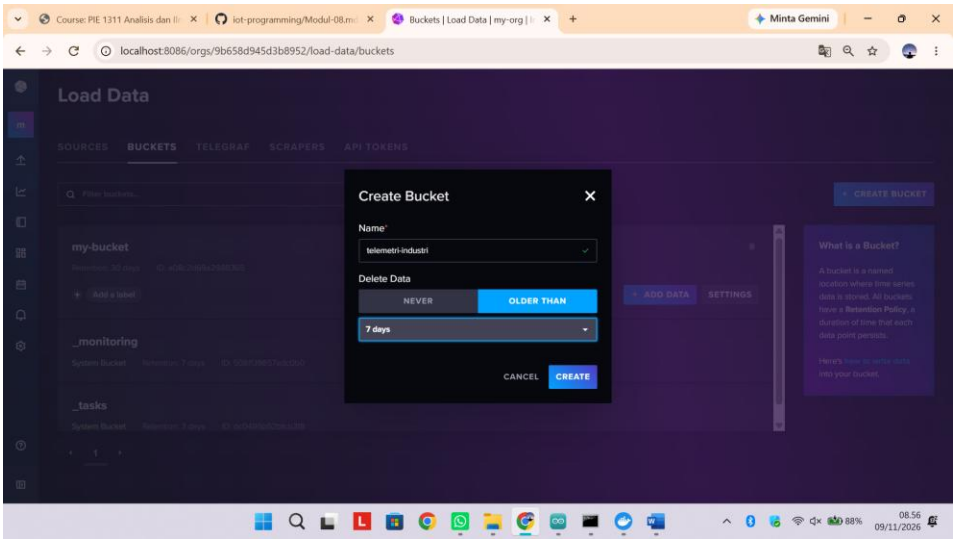
Dalam lingkungan industri nyata (seperti fasilitas pergudangan farmasi atau manufaktur elektronik berskala besar), sistem telemetri memantau lusinan ruangan dengan karakteristik termal yang berbeda-beda. Agar basis data dapat mengidentifikasi dan memisahkan aliran metrik secara akurat, seluruh paket data telemetri harus diperkaya dengan metadata kontekstual (*Multi-Zone Tagging*).

Instruksi Pengerjaan Langkah demi Langkah:

1. Pembuatan Bucket Khusus Industri di InfluxDB v2:

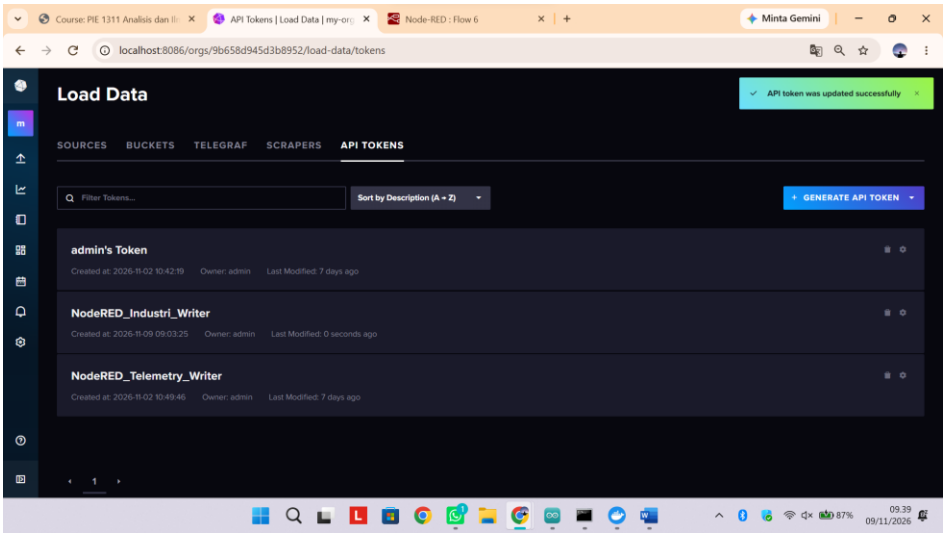
- Buka antarmuka InfluxDB di browser (http://localhost:8086).
- Masuk ke menu **Load Data > Buckets**.
- Klik tombol **Create Bucket** di sebelah kanan atas:
    - **Name:** telemetri-industri

- **Delete Data:** Pilih opsi **Older than**, lalu tentukan durasi retensi: **7 days** (7 hari).
- Klik **Create**.



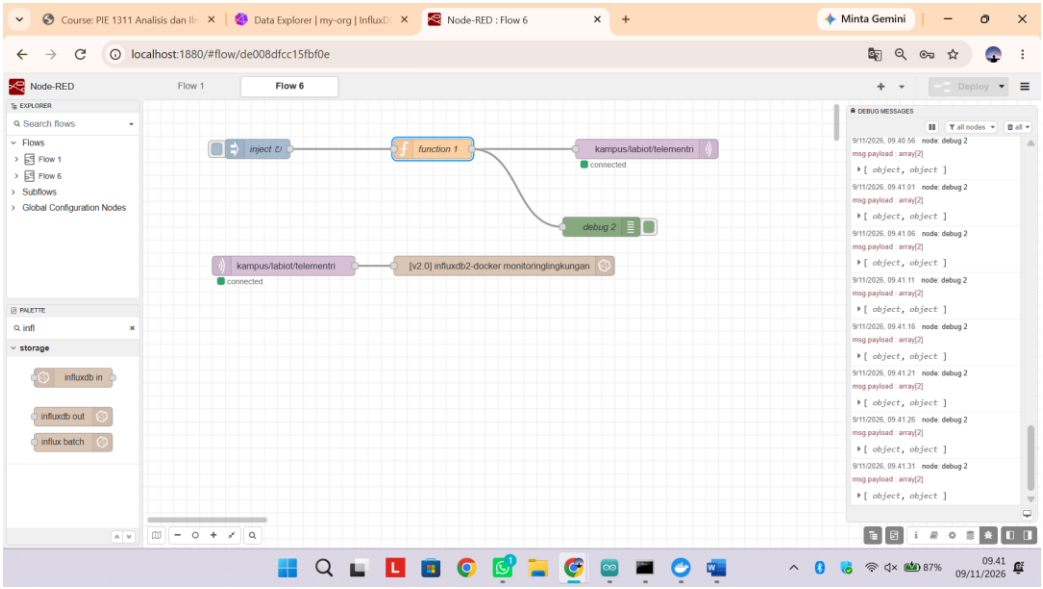
2. Pembaruan API Token Khusus:

- Masuk ke tab **API Tokens**, buat sebuah *Custom API Token* baru bernama NodeRED_Industri_Writer.
- Berikan izin **Write** khusus ke bucket baru telemetri-industri.
- Salin token tersebut untuk digunakan pada Node-RED.



3. Rekayasa Alur Kerja pada Node-RED:

- Buka kanvas Node-RED (http://localhost:1880).
- Perbarui node **function** agar dapat menerima nilai zona dan tipe_sensor secara dinamis dari payload JSON yang masuk, lalu memetakan variabel tersebut ke dalam objek *Tags* (elemen ke-2 array):
- Buka node **`influxdb out`**, perbarui konfigurasinya:
  - **Token:** Masukkan token baru `NodeRED_Industri_Writer`.
  - **Bucket:** Ganti dari `my-bucket` menjadi **`telemetri-industri`**.
  - **Measurement:** `monitoring_lingkungan`.
- Klik **Deploy**.



4. Modifikasi Program Mikrokontroler ESP8266:

- Buka Arduino IDE, modifikasi dokumen JSON pengiriman data telemetri pada skrip Praktikum 4:
  - Tambahkan identitas zona yang jelas: "zona": "Zona-A" (Jika bekerja dalam kelompok, minta rekan kelompok kedua untuk menyetel "zona": "Zona-B" dan clientId = "ESP8266_Local_Node02").
  - Unggah kode program ke NodeMCU ESP8266 dan amati aliran datanya pada Serial Monitor.

```
143     if (!isnan(t) && !isnan(h)) {
144       // Menyusun dokumen JSON telemetri
145       StaticJsonDocument<256> docOut;
146       docOut["node_id"]    = clientId;
147       docOut["suhu"]       = serialized(String(t, 2));
148       docOut["kelembapan"] = serialized(String(h, 2));
149       docOut["status_led"] = digitalRead(ledPin);
150       docOut["zona"]       = "Zona-A"; // Metadata penandaan zona ruang sensor
```

5. Perancangan Skrip Kueri Flux Analitik Lanjut:

- Buka kembali **Data Explorer** di InfluxDB (http://localhost:8086).
- Beralih ke mode **Script Editor** (tombol di kanan atas Query Builder).
- Tuliskan kueri Flux kustom untuk melakukan analisis berikut:
    i. Mengambil data dari bucket telemetri-industri.
    ii. Mengatur rentang waktu pencarian data selama 15 menit terakhir (range(start: -15m)).
    iii. Memfilter hanya data yang memiliki tag **zona == "Zona-A"**.
    iv. Memfilter hanya metrik **_field == "suhu"**.
    v. Menambahkan filter kondisi kritis: hanya tampilkan titik data yang memiliki nilai suhu melampaui **30.0∘C** (filter(fn: (r) => r._value > 30.0)).
- Klik tombol **Submit** dan amati hasil visualisasinya! Simpan skrip Flux yang Anda buat untuk dicantumkan ke dalam laporan praktikum.

